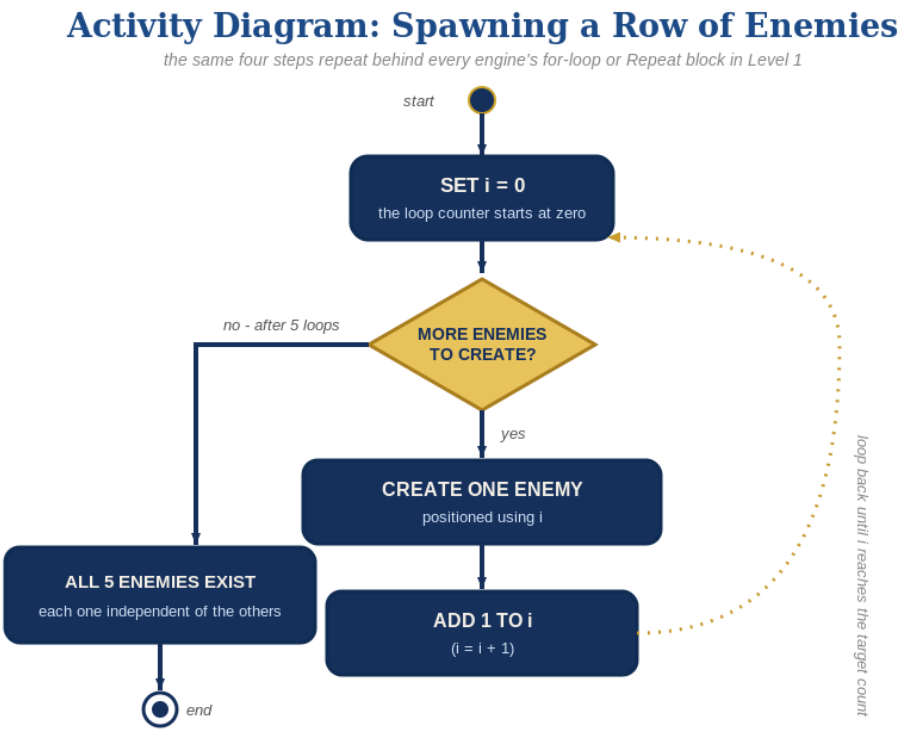
Lesson 5 Teacher's Key

Unreal

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

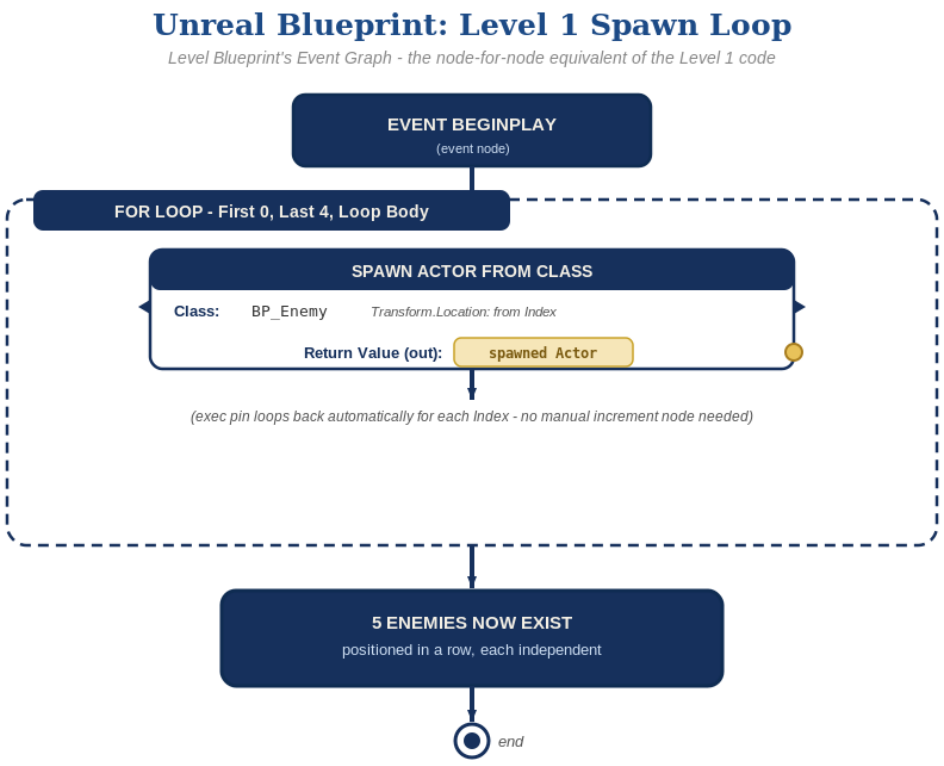


Every engine's Level 1 solution runs this same shape - a counter, a repeat check, one creation step, and an increment - even though the exact blocks or code syntax differ. Tracing a spawn bug means finding which of these four steps broke.

The spawn loop, in general - the same counter, repeat check, creation step, and increment behind Level 1 in every engine, whatever the exact blocks or syntax look like.

A For Loop node in the Level Blueprint - BP_Enemy needs zero changes from Lesson 4:

```
Level Blueprint:
Event BeginPlay ->
  For Loop(First Index = 0, Last Index = 4) ->
    Spawn Actor from Class(BP_Enemy, Transform.Location = (BaseX + Index *
150, BaseY, 0))
```



The Level Blueprint's Event Graph, node by node - the node-for-node equivalent of the Level 1 nodes above.

Level 2 - Complete Worked Solution

BP_Enemy's own Event ActorBeginOverlap and Cast To BP_Bullet logic is fully self-contained per instance, so every Actor the loop spawns behaves independently and correctly with no extra nodes needed for Level 2.

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking.

Sound: already works

The Play Sound 2D node inside BP_Enemy's overlap event runs independently for every enemy that gets hit - no changes needed.

Visual: striped row

Add a custom Event (e.g. SetStripeColor) to BP_Enemy that creates its dynamic material instance with a passed-in color, then call it from the Level Blueprint's loop right after each Spawn Actor from Class, alternating colors based on Index. Build Dark Red and Light Red with a Make Linear Color node, not Make Color - Set Vector Parameter Value expects a Linear Color, whose R/G/B fields are 0-1 floats, not Make Color's 0-255 bytes: Dark Red = R 0.8, G 0.1, B 0.2 (a deep crimson red), Light Red = R 0.95, G 0.4, B 0.5 (a lighter dusty rose/pink) - the

same two shades used in this course's Unity/Godot tracks, just as a 0-1 float here instead of 0-255:

```
BP_Enemy: Custom Event SetStripeColor(Color) ->
  Create Dynamic Material Instance(Self's Material) -> Set Vector
Parameter Value("Color", Color)

Level Blueprint:
Dark Red = Make Linear Color(R=0.8, G=0.1, B=0.2)
Light Red = Make Linear Color(R=0.95, G=0.4, B=0.5)

Inside the For Loop:
Spawn Actor from Class(BP_Enemy, ...) -> (Return Value) ->
SetStripeColor(Index % 2 == 0 ? Dark Red : Light Red)
```

Structure: spawner vs. behavior stay separate

The Level Blueprint's For Loop only ever spawns and positions enemies - collision-response and hit-sound logic stay entirely inside BP_Enemy's own Event Graph.

Complete Level 3 solution (all 3 goals combined)

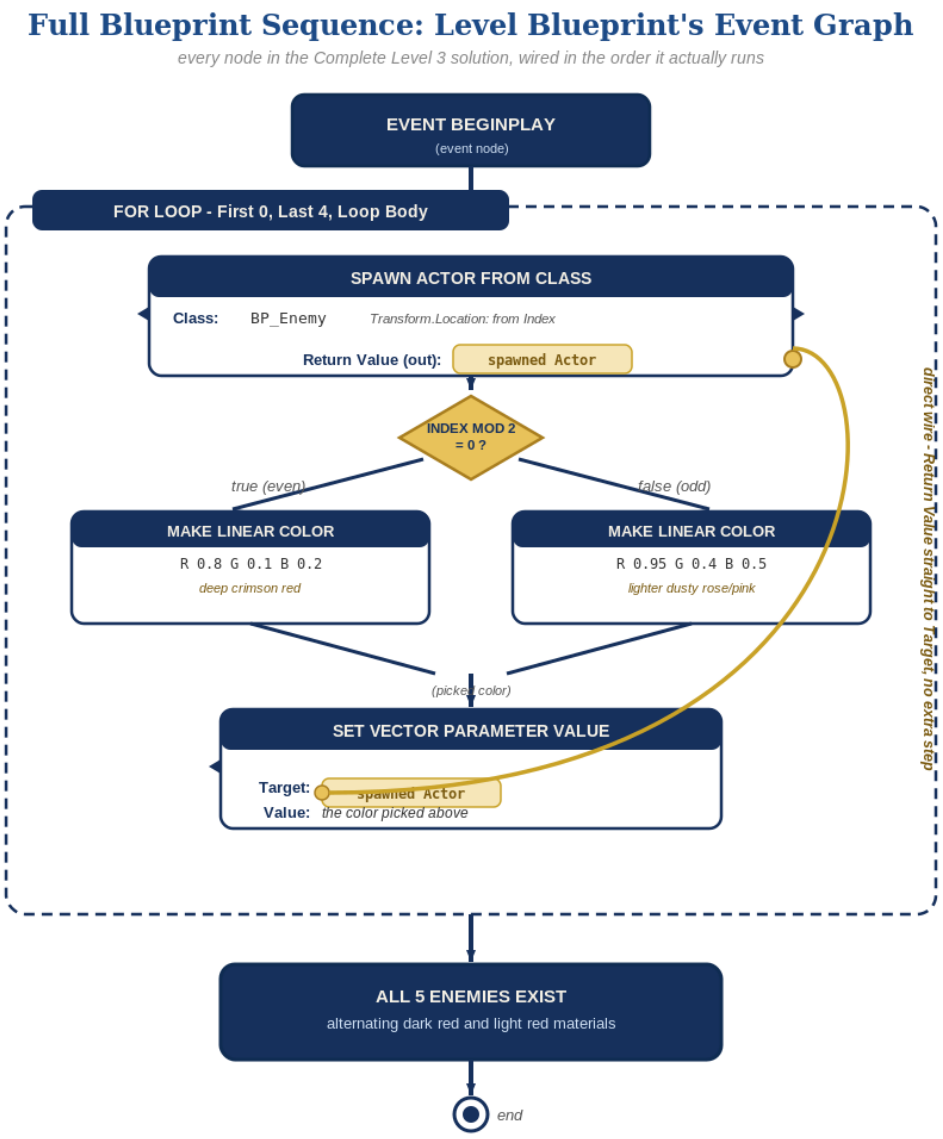
The spawn loop with the visual idea added - sound needs no changes since it already lives on BP_Enemy:

```
BP_Enemy: Custom Event SetStripeColor(Color) ->
  Create Dynamic Material Instance(Self's Material) -> Set Vector
Parameter Value("Color", Color)

Level Blueprint:
Dark Red = Make Linear Color(R=0.8, G=0.1, B=0.2)
Light Red = Make Linear Color(R=0.95, G=0.4, B=0.5)
Event BeginPlay ->
  For Loop(First Index = 0, Last Index = 4) ->
    Spawn Actor from Class(BP_Enemy, Transform.Location = (BaseX + Index *
150, BaseY, 0))
    -> (Return Value) -> SetStripeColor(Index % 2 == 0 ? Dark Red : Light
Red)
```

Complete Visual Sequence (all nodes combined)

Every node in the Complete Level 3 solution above, wired in the order it actually runs.



This is the node-for-node equivalent of the Complete Level 3 solution's Blueprint description above - Spawn Actor's Return Value pin feeds Set Vector Parameter Value directly, the same way Unity's Instantiate output does.

Spawn Actor from Class's own Return Value pin plugs straight into Set Vector Parameter Value's Target - the same direct-wire pattern as Unity's Instantiate, no promote-to-variable step needed.

Alternate Path Notes: C++

A plain C++ for loop calling GetWorld()->SpawnActor<ACppEnemy>() several times is the code equivalent of the Blueprint For Loop node.

The Level 3 visual idea maps onto a public method on ACppEnemy called right after each SpawnActor call - Sound needs no changes either way, since it lives on ACppEnemy's own OnOverlap(), not the spawner.

- Showing both C++ and Blueprint side by side for the same behavior keeps demystifying Blueprints for students who assume "visual" means "not real code" - it's the same engine calls either way.

- This is squarely the optional advanced track - most students should stay on Blueprints; C++ is for students already comfortable with typed languages from Godot, GameMaker, or Unity's C# path.